M22 Observability: JSONL & jq

Methodology Advanced

Auditing and analyzing Claude Code activity through logs

The JSONL registry: source of truth

Claude Code records conversation and tool activity in local JSONL transcripts stored in `~/.claude/projects/<project>/`. They help reconstruct activity, but recorded tool requests are not proof that every requested effect completed. Correlate tool results and external logs.

```
# Locate session files for the current project
ls ~/.claude/projects/
# Select the actual project directory; do not guess its encoding
# → SESSION_ID.jsonl  SESSION_ID2.jsonl ...
```

Each `assistant` type line contains a `content` array whose `tool_use` entries document each tool call with its input parameters.

What tools expose

Tool	Available information
`Read`	File path, line range
`Edit` / `Write`	Path, modified content
`Bash`	Full command executed
`WebFetch`	Requested URL and processing prompt
`Agent`	Prompt sent to sub-agent
`Glob` / `Grep`	Search patterns and scope

Essential jq queries

```
SESSION=~/.claude/projects/-my-project-/SESSION.jsonl

# Files read in the session
jq 'select(.type=="assistant") |
  .message.content[]? |
  select(.type=="tool_use" and .name=="Read") |
  .input.file_path' "$SESSION"

# Bash commands executed
jq 'select(.type=="assistant") |
  .message.content[]? |
  select(.type=="tool_use" and .name=="Bash") |
  .input.command' "$SESSION"

# Count by tool (activity profile)
jq -r 'select(.type=="assistant") |
  .message.content[]? |
  select(.type=="tool_use") | .name' \
  "$SESSION" | sort | uniq -c | sort -rn
```

Sensitive patterns to monitor

These patterns are worth detecting automatically in audits:

Pattern	Risk	jq filter
Read on `.env`, `*.pem`, `id_rsa`	Credentials access	`test("\\.(env|pem|key)$")`
Bash with `rm -rf` or `--force-push`	Destructive action	`test("rm -rf")`
WebFetch to external URL	Potential exfiltration	select `.name=="WebFetch"`
Write outside project directory	Scope creep	Compare path to current directory

For teams, export reviewed logs to access-controlled storage with an explicit retention policy. S3 versioning alone does not make logs immutable; that requires additional controls such as configured Object Lock.

Community external tools

Tool	Type	Primary usage
ccusage	CLI / TUI	Costs from JSONL, community reference
claude-code-otel	OpenTelemetry	Export to Prometheus + Grafana
ccboard	TUI + Web	Sessions dashboard, costs, activity
Akto	SaaS	Security guardrails at API level
MLflow Tracing	CLI + SDK	Exact token counts, LLM evaluation

```
# Quick install
npm i -g ccusage        # Costs and statistics
cargo install ccboard   # Interactive dashboard
# Native OpenTelemetry configuration:
# https://code.claude.com/docs/en/monitoring-usage
```

Decision tree

```
Need costs quickly?      → ccusage (0 config)
Enterprise audit?        → claude-code-otel + Grafana
Already on MLflow?       → MLflow tracing integration
Visual dashboard?        → ccboard
```

Cost estimation in JSONL

JSONL logs do not contain Anthropic prices directly. Tools such as ccusage read recorded usage counters and apply model pricing to estimate cost. Missing records, pricing changes and subscription billing can make those estimates differ from the provider invoice.